

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

LAUNDRYMATE

Absolvent
Zamfirescu Alexandra

Coordonator științific
Conf. univ. dr. Ciprian Ionuț Păduraru

București, iunie 2025

Rezumat

Într-o lume în care timpul constituie o resursă tot mai prețioasă, iar eficiența a devenit o necesitate cotidiană, activitățile casnice precum spălatul hainelor sunt adesea realizate în grabă, fără a se acorda atenția necesară instrucțiunilor de întreținere. Mulți utilizatori nu sunt familiarizați cu semnificația simbolurilor de pe etichetele textile, ceea ce determină configurarea incorectă a programului mașinii de spălat. Acest lucru poate duce la uzura hainelor, risipă de resurse și chiar la reducerea duratei de viață a echipamentului.
LaundryMate este o aplicație concepută pentru a ghida utilizatorii în înțelegerea și recunoașterea simbolurilor de spălare de pe etichetele hainelor, oferindu-le sprijin în procesul de grupare corectă a acestora. Prin integrarea unui sistem de recunoaștere vizuală automată, aplicația identifică simbolurile textile și recomandă un program de spălare adecvat, contribuind astfel la protejarea articolelor vestimentare și la o utilizare mai eficientă a mașinii de spălat.

Abstract

In a world where time is an increasingly valuable resource and efficiency had become a daily necessity, household activities such as doing laundry are often carried out in haste, with little attention paid to care label instructions. Many users are unfamiliar with the meaning of textile care symbols, which often leads to incorrect washing machine settings. This can result in damaged garments, wasted resources, and even reduced appliance lifespan. **LaundryMate** is a mobile application designed to assist users in understanding and interpreting the care symbols found on clothing labels, while also helping them properly group garments for washing. By integrating an automated visual recognition system, the application identifies the care symbols specific to each item and recommends a suitable washing program, thereby contributing to garment protection and more efficient use of washing machine.

Cuprins

1 Introducere **6**

1.1 Contextul și motivația 6

1.2 Obiective 6

1.2.1 Recunoașterea simbolurilor de întreținere 7

1.2.2 Construirea unui program de spălare standardizat 7

1.2.3 Istoric al hainelor scanate 7

1.2.4 Recomandări zilnice cu LLM (Large Language Model) 7

1.2.5 Notificări periodice pentru utilizatori 7

1.3 Structura lucrării 7

2 Analiza soluțiilor existente **9**

2.1 Care Label Recognition using Scene Text Reading 9

2.2 Information Extraction from Product Labels: A Machine Vision Approach 9

2.3 Developing an Intelligent Washing Machine with a Fuzzy Logic Controller 10

2.4 Aplicații mobile și soluții comerciale existente 10

3 Tehnologii utilizate **12**

3.1 Interfața mobilă 12

3.1.1 React Native 12

3.1.2 Expo 12

3.2 Backend-ul 12

3.2.1 Django și Django REST Framework 12

3.2.2 Celery și Celery Beat 13

3.3 Autentificarea și stocarea datelor 13

3.3.1 Firebase 13

3.4 Modelul de detecție vizuală 13

3.4.1 YOLO 13

3.5 Modelul LLM 13

3.5.1 Gemini 13

4 Implementare **14**
4.1 Arhitectura generală a sistemului 14
4.2 Componentele backend-ului 15
4.2.1 Endpoint-uri REST și fluxul cererilor 15
4.2.2 Integrarea serviciilor externe 15
4.2.3 Trimiterea notificărilor periodice 16
4.3 Componentele frontend-ului 17
4.3.1 Navigarea între ecrane și sistemul de rutare 17
4.3.2 Interacțiunea cu backend-ul 17
4.4 Modelul de detecție vizuală 18
4.4.1 Descrierea generală a arhitecturii YOLO 18
4.4.2 Construirea datasetului și pre-procesarea imaginilor 18
4.4.3 Antrenarea modelelor 19
4.4.4 Rezultate 19
4.4.5 Interacțiunea cu backend-ul 20
4.5 Logica deterministă de generare a recomandărilor 20

5 Fluxul de utilizare al aplicației **22**
5.1 Autentificarea - pagina de înregistrare și logare 22
5.2 Gestionarea contului și preferințelor utilizatorului 23
5.3 Pagina principală 23
5.4 Scanarea hainelor și generarea recomandării 24
5.5 Salvarea hainelor în istoric 24
5.6 Notificări și reamintiri automate 24

6 Concluzii **25**

Bibliografie **26**

Listă de figuri

4.1 Arhitectura aplicației LaundryMate 14
4.2 Integrarea LLM în backend 16
4.3 Apelul către endpoint-ul /chat/ 18
4.4 Afișarea răspunsului în interfață 18
4.5 Curba Precision-Recall pentru YOLOv5s mAP@0.5 = 0.758 19
4.6 Curba Precision-Recall pentru YOLOv8n mAP@0.5 = 0.804 19
4.7 Matricea de confuzie pentru YOLOv5s 20
4.8 Matricea de confuzie pentru YOLOv8n 20
4.9 Trimiterea imaginii către backend 20
4.10 Algoritmul de grupare 21

5.1 Autentificarea în aplicație 22
5.2 Meniul de preferințe al utilizatorului 23
5.3 Pagina principală 23
5.4 Fluxul de scanare și afișare 24
5.5 Exemplu de notificare primită de utilizator 24

Capitolul 1

Introducere

1.1 Contextul și motivația

Această lucrare pornește de la o experiență personală întâmpinată frecvent: utilizarea pentru prima dată a unei mașini de spălat, fără cunoștințe prealabile despre simbolurile de întreținere sau despre gruparea corectă a hainelor. În acest context, **LaundryMate** este conceput ca o aplicație mobilă capabilă să recunoască simbolurile de întreținere și să ofere un program de spălare adecvat. Prin integrarea tehnologiilor de detecție vizuală și logică decizională, aplicația urmărește sprijinirea utilizatorilor în alegerea optimă a unui ciclu de spălare, contribuind totodată la reducerea consumului de resurse și la protejarea materialelor textile.

1.2 Obiective

Lucrarea urmărește dezvoltarea unei aplicații mobile care asistă utilizatorii la procesul de sortare și configurare a spălării hainelor, pe baza simbolurilor de pe etichete. Principalele obiective sunt:

- Recunoașterea simbolurilor de îngrijire de pe etichetele hainelor folosind un model de vedere automată;
- Construirea unui program determinist de spălare, independent de modelul mașinii de spălat;
- Salvarea hainelor scanate într-un istoric personal al utilizatorului;
- Furnizarea de sfaturi de îngrijire zilnice prin integrarea unui model LLM;
- Implementarea unui sistem de notificare periodică pentru susținerea utilizării constante a aplicației.

1.2.1 Recunoașterea simbolurilor de întreținere

Utilizând modelul de detecție **YOLOv8n**, aplicația identifică automat simbolurile de pe etichete, eliminând necesitatea interpretării manuale și asigurând o extragere precisă a informațiilor relevante.

1.2.2 Construirea unui program de spălare standardizat

Aplicația permite utilizatorului să specifice manual materialul și culoarea fiecărui articol vestimentar, în timp ce simbolurile de întreținere sunt detectate automat. Pe baza acestor informații, sistemul urmează o logică deterministă construită dintr-un set de reguli, cu scopul de a grupa hainele compatibile și de a genera un program de spălare optim.

1.2.3 Istoric al hainelor scanate

Pentru a facilita reutilizarea informațiilor și a reduce scanările repetitive, aplicația stochează articolele vestimentare scanate și grupurile de haine trimise la spălare într-un istoric asociat fiecărui utilizator. Fiecare haină este salvată cu simbolurile detectate și data scanării, iar grupurile sunt înregistrate împreună cu programul de spălare generat. Această funcționalitate permite consultarea rapidă a recomandărilor anterioare și simplifică deciziile viitoare de spălare.

1.2.4 Recomandări zilnice cu LLM (Large Language Model)

Aplicația utilizează modelul de limbaj **Gemini 2.0 Flash**, integrat prin intermediul API-ului Google Generative AI, pentru generarea automată a unor recomandări zilnice, afișate sub forma unui "Tip of the Day". Acestea oferă utilizatorului informații succinte și relevante privind întreținerea corectă a hainelor.

1.2.5 Notificări periodice pentru utilizatori

Implementarea sistemului de notificare periodică are ca scop reamintirea utilizatorilor de a scana hainele și de a utiliza aplicația în mod constant. Notificările sunt declanșate automat cu ajutorul unui task recurent utilizând **Celery Beat**.

1.3 Structura lucrării

Conținutul lucrării este împărțit în șase capitole, care urmăresc progresiv dezvoltarea aplicației LaundryMate, de la contextul inițial până la concluzii.

Capitolul 2 oferă o sinteză a lucrărilor relevante, analizând atât soluții academice, cât și aplicații mobile existente. Acestea oferă un cadru comparativ util pentru evidențierea specificului adus de LaundryMate.

Capitolul 3 prezintă tehnologiile utilizate în dezvoltarea aplicației, acoperind interfața mobilă, componenta backend și integrarea inteligenței artificiale în aplicație.

Capitolul 4 descrie arhitectura sistemului și procesul de implementare, incluzând organizarea modulară a codului, integrarea modelului YOLO[1], gestionarea simbolurilor detectate și logica de generare a programului de spălare.

Capitolul 5 ilustrează fluxul de utilizare al aplicației, alături de principalele ecrane și funcționalități implementate. Prin intermediul capturilor de ecran, sunt evidențiate modul de funcționare al aplicației și comportamentul acesteia în situații concrete, precum scanarea simbolurilor, generarea programului de spălare și salvarea în istoricul utilizatorului.

Capitolul 6 încheie lucrarea prin evidențierea principalelor realizări și a perspectivelor de îmbunătățire a aplicației LaundryMate.

[1]YOLO = You Only Look Once

Capitolul 2

Analiza soluțiilor existente

2.1 Care Label Recognition using Scene Text Reading

Lucrarea abordează problema recunoașterii etichetelor într-un cadru real, propunând o arhitectură E2E-MLT (End-to-End Multi-Lingual Text) care identifică atât textul, cât și simbolurile de îngrijire prezente pe etichete. Se pune accentul pe corelația dintre simbolurile de întreținere și textul descriptiv al etichetelor, obținându-se o acuratețe de până la 96.8% în identificarea compoziției materialelor [1]. Metoda implică o detecție în două faze: întâi sunt localizate regiunile de interes, urmate de clasificarea simbolurilor și a textului, fiind testată pe un set de date restrâns.

Spre deosebire de această abordare, aplicația LaundryMate utilizează un model YOLOv8n, antrenat pe un set de date mai extins și personalizat. Alegerea YOLO este justificată prin rapiditatea inferenței, precizia în detecția simbolurilor și eficiența sa în rularea pe dispozitive mobile. În plus, modelul permite o detecție unică, în timp real, fără a necesita etape separate de clasificare, ceea ce îl face mult mai potrivit pentru integrarea într-o aplicație mobilă.

2.2 Information Extraction from Product Labels: A Machine Vision Approach

Această lucrare recentă propune un sistem automat de a extrage informații de pe etichetele produselor comerciale. Soluția combină detecția obiectelor și OCR[1], construind o reprezentare semantică a celor mai importante elemente de pe o etichetă, cum ar fi ingredientele, avertismentele sau instrucțiunile de utilizare [2]. Deși acest tip de abordare este eficient în cazul produselor de uz casnic, aplicarea sa pe etichetele textile poate fi problematică din cauza variabilității fonturilor, dimensiunii textului și uzurii fizice a etichetelor.

[1]OCR = Optical Character Recognition

În aplicația LaundryMate, accentul este pus pe detecția simbolurilor standardizate de întreținere, care sunt mai vizibile și clare decât textul. Din acest motiv, s-a ales excluderea componentei OCR și utilizarea exclusivă a unui model de detecție vizuală.

2.3 Developing an Intelligent Washing Machine with a Fuzzy Logic Controller

Printre soluțiile inteligente de automatizare a ciclurilor de spălare se numără și cele bazate pe **logica fuzzy**, un model computațional inspirat din raționamentul uman care permite luarea deciziilor în condiții de incertitudine. Aceste sisteme folosesc structuri decizionale, ajustând automat parametri precum durata unui ciclu, temperatura și cantitatea de detergent, în funcție de factori precum gradul de murdărie, tipul țesăturii sau culoarea hainelor [3]. Prin aplicarea a sute de reguli fuzzy, sistemul optimizează procesul de spălare și reduce consumul de resurse.

Spre deosebire de aceste sisteme integrate direct în hardware-ul mașinii, LaundryMate aplică o abordare software complementară. Simbolurile de întreținere detectate vizual sunt procesate printr-o logică decizională deterministă, cu scopul de a îndruma utilizatorul în alegerea programului potrivit.

2.4 Aplicații mobile și soluții comerciale existente

Laundry Symbols AI Scanner

Laundry Symbols AI Scanner este o aplicație mobilă disponibilă în App Store, care utilizează inteligența artificială pentru identificarea simbolurilor de pe etichetele hainelor [4]. Deși oferă un nivel de asistență vizuală pentru interpretarea simbolurilor, aplicația se limitează la afișarea semnificației acestora și nu integrează funcționalități suplimentare precum gruparea hainelor sau recomandarea programului de spălare.

Pe de altă parte, LaundryMate extinde acest concept prin integrarea unui sistem de detecție în timp real, a unui istoric al hainelor și a unei logici decizionale care ghidează utilizatorul în alegerea unui program cât mai potrivit.

Apple Vision (iOS 17+)

Începând cu iOS 17, funcția **Visual Look Up** integrată de iOS permite atât recunoașterea martorilor din bordul mașinii, cât și simbolurile de îngrijire de pe etichetele articolelor vestimentare. Utilizatorii pot accesa poza din galerie, ținând apăsat pe imagine, iar sistemul afișează informații pentru simbolurile detectate. Această opțiune este complet integrată în sistemul de operare și utilizează capabilități de inteligență artificială on-device

ale Apple, garantând confidențialitate și o experiență fluidă [5]. Totuși, deși oferă o traducere vizuală a simbolurilor, nu permite sortarea hainelor sau recomandarea unor programe personalizate de spălare.

Astfel, spre deosebire de LaundryMate, care integrează un flux complet, Apple Vision are un rol informativ și static, fără interacțiune cu procesele de spălare.

Capitolul 3

Tehnologii utilizate

3.1 Interfața mobilă

3.1.1 React Native

Pentru dezvoltarea interfeței mobile a aplicației s-a utilizat framework-ul **React Native**, având avantajul dezvoltării simultane pentru Android și iOS dintr-o singură bază de cod JavaScript [6]. Aplicația este structurată modular, fiecare ecran funcțional fiind implementat ca o componentă React separată. Navigarea între ecrane este gestionată cu ajutorul bibliotecii `react-navigation`, iar stilizarea este realizată utilizând componente native React Native.

3.1.2 Expo

Expo este un framework care extinde React Native și simplifică dezvoltarea aplicațiilor mobile, oferind un mediu de testare rapid și suport pentru funcționalități precum camera, notificările și rularea directă pe dispozitivele reale prin aplicația **Expo Go** [7].

3.2 Backend-ul

3.2.1 Django și Django REST Framework

Componenta server a aplicației a fost implementată cu ajutorul framework-ului Django, ales pentru simplitatea configurării și suportul pentru integrarea modelelor de învățare automată. Prin extensia **Django REST Framework**, au fost definite endpoint-uri REST[1] care permit comunicarea cu aplicația mobilă, utilizând formatul JSON[2] pentru transmiterea datelor [8].

[1]REST = Representational State Transfer
[2]JSON = JavaScript Object Notation

3.2.2 Celery și Celery Beat

Pentru trimiterea notifcărilor periodice, aplicația folosește **Celery** și **Celery Beat**, care permit rularea asincronă a task-urilor planificate [9]. În LaundryMate, acest sistem verifică zilnic activitatea utilizatorilor și trimite notificări celor care nu au scanat haine în ultimele 3 zile.

3.3 Autentificarea și stocarea datelor

3.3.1 Firebase

Firebase este o platformă dezvoltată de Google, dedicată aplicațiilor mobile și web, oferind servicii scalabile de backend. În cadrul aplicației au fost utilizate două componente principale: **Firebase Authentication**, pentru gestionarea autentificării utilizatorilor prin e-mail și parolă, și **Firebase Firestore**, pentru stocarea în cloud a datelor asociate fiecărui utilizator, precum hainele scanate sau grupurile de spălare. Firestore este o bază de date NoSQL optimizată pentru aplicații mobile, oferind sincronizare în timp real și integrare facilă în codul frontend, ceea ce a reprezentat un criteriu important în alegerea acestei soluții [10].

3.4 Modelul de detecție vizuală

3.4.1 YOLO

Detectarea simbolurilor de pe etichetele hainelor a fost realizată cu ajutorul unui model din familia **YOLO** (You Only Look Once), cunoscut pentru eficiența sa în sarcini de detecție a obiectelor în timp real [11]. Dintre versiunile analizate, a fost selectat modelul **YOLOv8n**[3], datorită echilibrului între dimensiune, viteză de inferență și acuratețe, fiind potrivit pentru aplicații mobile [12]. Setul de date utilizat a fost construit manual, iar adnotările au fost realizate cu ajutorul platformei **Roboflow** [13].

3.5 Modelul LLM

3.5.1 Gemini

Aplicația integrează **Gemini 2.0 Flash**, un model LLM dezvoltat de Google, utilizat pentru generarea automată a sugestiilor zilnice [14]. Comunicarea cu modelul se realizează printr-un API REST, iar răspunsurile sunt procesate și afișate în secțiunea ”Tip of the Day”.

[3]YOLOv8n = versiunea 8 a modelului YOLO, varianta nano

Capitolul 4

Implementare

4.1 Arhitectura generală a sistemului

În ceea ce privește arhitectura aplicației LaundryMate, aceasta este structurată în patru componente principale: **frontend**, **backend**, **serviciile de infrastructură** (Firebase) și **serviciile externe de inteligență artificială**.

Frontend-ul, dezvoltat cu React Native și gestionat prin Expo, permite utilizatorului să interacționeze cu aplicația pentru scanarea etichetelor, vizualizarea recomandărilor sau gestionarea istoricului.

Backend-ul este implementat cu Django REST, expunând endpoint-uri pentru procesarea imaginilor și comunicarea cu serviciile externe.

Infrastructura Firebase oferă autentificare (Firebase Authentication) și stocare de date în cloud (Firestore) iar componentele AI externe includ serviciul Roboflow, pentru detecția simbolurilor de întreținere, și Gemini, utilizat pentru generarea sugestiilor zilnice.

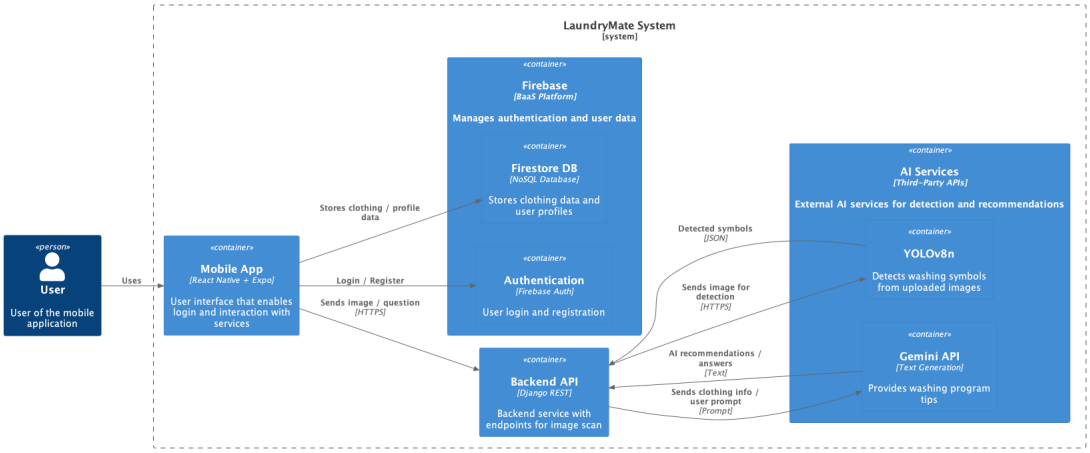


Figura 4.1: Arhitectura aplicației LaundryMate

4.2 Componentele backend-ului

Arhitectura backend-ului este modulară , fiind organizată în directoare specifice pentru fiecare responsabilitate:

```
lm_backend/
  api/                # endpoint-uri REST pentru scanare, LLM, notificari
  credentials/        # chei si configurari sensibile
  django_backend/     # configurarea generala Django si Celery
  media/              # imagini incarcate sau procesate
  model/              # integrarea modelului YOLOv8n
  requirements.txt    # dependente
  run_local.sh        # script de pornire locala
```

4.2.1 Endpoint-uri REST și fluxul cererilor

În backend sunt definite mai multe endpoint-uri REST prin intermediul Django REST Framework, fiecare fiind asociat unei funcționalități principale a aplicației.

Imaginile trimise de utilizator sunt gestionate prin endpoint-ul `/upload/` și procesate cu ajutorul modelului YOLOv8n prin `/detect-symbols/`, ale cărui rezultate sunt analizate de un modul decizional care stabilește programul potrivit de spălare.

Istoricul hainelor este accesat prin endpoint-ul `/garments/`, iar interacțiunea cu modelul LLM pentru generarea recomandărilor zilnice se realizează prin rutele `/chat/`.

Trimiterea notificărilor automate este gestionată prin `/notifications/`.

Răspunsurile sunt returnate în format JSON, permițând o integrare facilă cu frontend-ul și o procesare rapidă a datelor primite.

4.2.2 Integrarea serviciilor externe

Firebase

Așa cum a fost menționat și în **Capitolul 2 - Tehnologii utilizate**, Firebase este folosit în aplicația LaundryMate pentru două funcționalități esențiale: autentificarea utilizatorilor și stocarea datelor în cloud.

Autentificarea este gestionată de serviciul **Firebase Authentication**, care permite înregistrarea, verificarea adresei e-mail la crearea contului și logarea utilizatorilor în aplicație. Integrarea se face direct din aplicația mobilă, utilizând SDK-ul Firebase pentru React Native.

Pentru persistența datelor, aplicația utilizează **Firebase Firestore**, o bază de date NoSQL flexibilă și scalabilă. Structura bazei de date este centrată pe colecția `/users/`, fiecare document reprezentând unui utilizator autentificat. La rândul său, fiecare utilizator

are o colecție de haine individuale, precum `/garments/` și de grupurile de haine trimise la spălat salvate în `/groups/`.

Accesul la date este controlat prin reguli de securitate Firebase, care permit doar utilizatorilor autentificați să acceseze propriile date.

Gemini 2.0 Flash

Pentru generarea recomandărilor zilnice, aplicația integrează modelul **Gemini 2.0 Flash** oferit de Google. Backend-ul gestionează cererile utilizatorului printr-un endpoint asociat, după următorul flux:

- Se primește un request POST la endpoint-ul `/chat/` cu un mesaj în corpul cererii
- Se încarcă cheia API Gemini din fișierul `.env`
- Se configurează clientul Gemini folosind biblioteca `google.generativeai`
- Se creează o instanță a modelului `gemini-2.0-flash`
- Se trimite promptul primit către LLM
- Răspunsul generat va fi primit și returnat ca JSON către frontend

```python
class ChatAPIView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        user_message = request.data.get("message", "")

        if not user_message:
            return Response({"error": "No message provided."}, status=status.HTTP_400_BAD_REQUEST)

        try:
            genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

            model = genai.GenerativeModel("gemini-2.0-flash")
            response = model.generate_content(user_message)

            reply = response.text
            return Response({"reply": reply})
        except Exception as e:
            return Response({"error": str(e)}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)
```

Figura 4.2: Integrarea LLM în backend

4.2.3 Trimiterea notificărilor periodice

Pentru trimiterea automată a notificărilor, aplicația utilizează biblioteca **Celery**, împreună cu extensia **Celery Beat**. Aceasta este configurată în proiectul Django cu un broker **Redis**, iar task-urile sunt programate cu Celery Beat pentru a rula zilnic la ora 10:00. Rularea acestora este separată de serverul principal și se realizează prin comenzi dedicate. Un task backend verifică activitatea utilizatorilor în baza de date Firestore, iar în cazul în care nu au fost înregistrate haine scanate în ultimele 3 zile, trimite o notificare push prin framework-ul **Expo**. Interacțiunea cu frontend-ul este realizată printr-un request POST către endpoint-ul `/notifications/`.

4.3 Componentele frontend-ului

Structura frontend-ului este, de asemenea, modulară, având directoare separate pentru ecrane, componente și funcționalități auxiliare.

```
lm_frontend/
  app/                    # ecrane principale si rutare
  components/             # elemente UI reutilizabile
  context/                # gestiune tema (light/dark)
  constants/              # palete de culori, dimensiuni etc.
  hooks/                  # functii personalizate pentru logica aplicatiei
  src/                    # apeluri catre backend
  translations/           # suport pentru mai multe limbi (ro/en)
  firebaseConfig.ts       # initializare Firebase
  App.tsx                 # componenta root
  i18n.ts                 # configurare traduceri
  app.json                # setari generale Expo
```

4.3.1 Navigarea între ecrane și sistemul de rutare

Navigarea este gestionată prin biblioteca `Expo Router` care extinde funcționalitatea `react-navigation` și permite organizarea ecranelor în structură de directoare.

În directorul `app/` se găsesc principalele secțiuni ale aplicației:

```
app/
 (tabs)/           # ecranele principale: Home, Scan, History, Profile
 auth/             # ecranele de autentificare si verificare cont
```

Navigarea este definită prin fișierele `_layout.tsx`, care specifică rutele pentru fiecare secțiune. Expo Router generează automat rutele pe baza structurii directoarelor din `app/`, însă logica de redirecționare este implementată explicit în cod. Interfața este organizată în tab-uri poziționate în partea de jos a ecranului, iar ecranele de autentificare sunt accesibile doar înainte de logare.

4.3.2 Interacțiunea cu backend-ul

Aplicația mobilă comunică cu backend-ul prin cereri HTTP, folosind funcții asincrone direct din fișierele de ecran.

LLM

- Pentru afișarea recomandării zilnice oferite de LLM pe pagina principală, funcția `getLaundryTip()` trimite un mesaj către endpoint-ul `/chat/`. Răspunsul generat

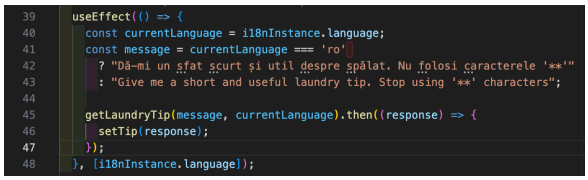
de modelul Gemini este afișat apoi ca ”Tip of the Day” în interfață.



```
export async function getLaundryTip(message: string, language: string = 'en') {
  try {
    const response = await fetch("http://192.168.100.113:8000/api/chat/", { // modificare ip manual
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        message,
        language
      }),
    });

    const data = await response.json();
    return data.reply;
  } catch (error) {
    console.error("Error fetching chat response:", error);
    return language === 'ro'
      ? "Ups! Ceva nu a mers bine."
      : "Oops! Something went wrong.";
  }
}
```

Figura 4.3: Apelul către endpoint-ul /chat/



```
useEffect(() => {
  const currentLanguage = i18nInstance.language;
  const message = currentLanguage === 'ro'
    ? "Dă-mi un sfat scurt și util despre spălat. Nu folosi caracterele '**'"
    : "Give me a short and useful laundry tip. Stop using '**' characters";

  getLaundryTip(message, currentLanguage).then((response) => {
    setTip(response);
  });
}, [i18nInstance.language]);
```

Figura 4.4: Afișarea răspunsului în interfață

Salvarea în istoric

- În ceea ce privește salvarea hainelor scanate și a grupurilor de spălare, aplicația transmite obiectele detectate către un endpoint-ul `/garments/`, salvându-le în Firestore în colecțiile specifice fiecărui utilizator. Aceste date sunt afișate pe pagina de istoric și reutilizate pentru recomandări viitoare.

Trimiterea notificărilor

- Pentru trimiterea notificărilor push, aplicația mobilă generează un `Expo Push Token` către backend, unde este procesat de endpoint-ul `/notifications/`. Acesta este utilizat pentru a construi un payload JSON și a trimite notificarea prin Expo.

4.4 Modelul de detecție vizuală

4.4.1 Descrierea generală a arhitecturii YOLO

Modelul utilizat pentru detecția simbolurilor este din familia YOLO (You Only Look Once), recunoscut pentru performanțele sale în timp real. YOLO transformă sarcina de detecție într-o problemă de regresie, împărțind imaginea în celule și precizând direct coordonatele obiectelor, clasele și scorul de încredere. Pentru LaundryMate, a fost ales modelul **YOLOv8n**, o versiune nano, optimizată pentru aplicații mobile, oferind un compromis bun între viteză și acuratețe.

4.4.2 Construirea datasetului și pre-procesarea imaginilor

Datasetul utilizat a fost construit în platforma Roboflow, pornind de la un set inițial de imagini adnotate manual, ce conțin simboluri standardizate de întreținere a textilelor. Setul de date a fost extins gradual prin colectarea de etichete suplimentare, ajungând la un total de peste 2200 de imagini în versiunea finală v7.

Pre-procesarea imaginilor a inclus redimensionarea la 416x416 pixeli și aplicarea de tehnici de îmbunătățire a contrastului (adaptive equalization). Pentru a îmbunătăți generalizarea modelului, au fost aplicate augmentări precum rotiri, flip, ajustarea luminozității și a saturației.

4.4.3 Antrenarea modelelor

Au fost testate două arhitecturi YOLO: **YOLOv5s** [15] și **YOLOv8n**, fiecare antrenat pe același set de date, timp de 50 de epoci, utilizând GPU local.

- **YOLOv5s**: versiunea "small" din arhitectura YOLOv5, cunoscută pentru stabilitate.
- **YOLOv8n**: versiunea "nano" a modelului YOLOv8, optimizată pentru mobil.

Modelul YOLOv8n a fost ales pentru integrarea finală în aplicație datorită performanțelor superioare.

4.4.4 Rezultate

Comparația performanței modelelor este prezentată mai jos, folosind metricile standard mAP@0.5, precizie și recall:

- **YOLOv5s**: mAP@0.5 = 0.758
- **YOLOv8n**: mAP@0.5 = 0.804

YOLOv8n a depășit YOLOv5s în majoritatea claselor, obținând scoruri superioare pentru simboluri precum **90C**, **DoNotTumbleDry** și **IronHigh**.

În schimb, YOLOv5s a avut un comportament mai stabil pe clasele **DryClean** și **IronMedium**.

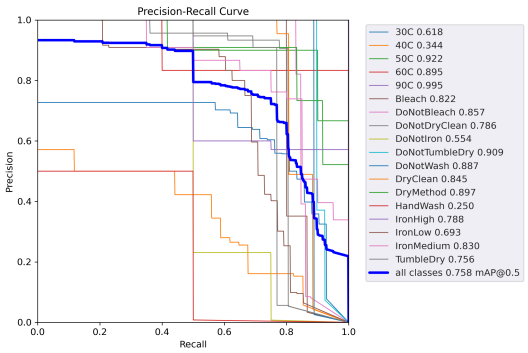


Figura 4.5: Curba Precision-Recall pentru YOLOv5s mAP@0.5 = 0.758

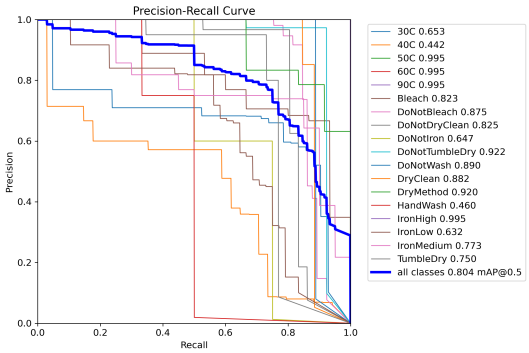


Figura 4.6: Curba Precision-Recall pentru YOLOv8n mAP@0.5 = 0.804

Analiza erorilor și matricea de confuzie

Una dintre cele mai frecvente confuzii apărute la ambele modele au fost între simbolurile de temperatură **30C** și **40C**, având forme vizuale similare și au fost adesea clasificate greșit unul în locul celuilalt. De asemenea, simbolurile **IronLow** și **IronMedium** au fost surse frecvente de erori, în special în cazul în care simbolurile erau parțial șterse sau neclare pe etichetă.

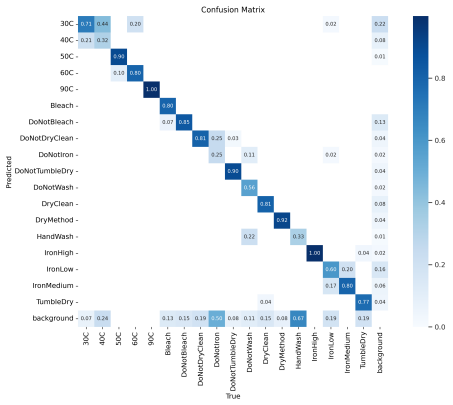


Figura 4.7: Matricea de confuzie pentru YOLOv5s

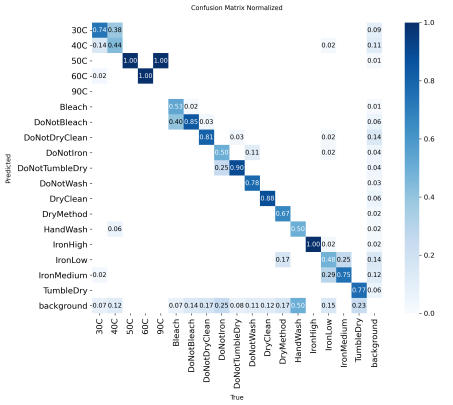


Figura 4.8: Matricea de confuzie pentru YOLOv8n

Modelul YOLOv8n s-a dovedit mai robust în fața acestor confuzii, înregistrând mai puține falsuri pozitive și negative, datorită capacității sale mai bune de generalizare.

4.4.5 Interacțiunea cu backend-ul

- În pagina de scanare, utilizatorul poate încărca sau fotografia o etichetă. Imaginea este trimisă către endpoint-ul `/detect-symbols/`, unde este procesată de modelul YOLOv8n. Simbolurile detectate sunt apoi analizate local pentru a extrage informații precum temperatura și tipul materialului.

```
112   const response = await fetch('http://192.168.100.113:8000/api/detect-symbols/', {
113     method: 'POST',
114     headers: {
115       'Content-Type': 'multipart/form-data',
116     },
117     body: formData,
118   });
```

Figura 4.9: Trimiterea imaginii către backend

4.5 Logica deterministă de generare a recomandărilor

După ce simbolurile de întreținere au fost detectate cu ajutorul modelului YOLOv8n, aplicația nu trimite aceste informații către un model AI, ci utilizează un set de **reguli**

deterministe pentru gruparea hainelor și sugerarea unui program de spălare corespunzător. Fiecare haină este caracterizată de următoarele atribute: temperatura maximă admisă la spălare, permisiunea de uscare automată și călcare, materialul precizat de utilizator și culoarea hainelor.

Procesul de generare a recomandării

1. **Extragerea atributelor relevante**
   Simbolurile detectate sunt convertite în atribute interpretabile, iar utilizatorul introduce manual materialul și culoarea.

2. **Definirea unui proces de spălare**
   Fiecărui articol i se asociază un profil unic de spălare, care include toate atributele relevante pentru grupare.

3. **Gruparea hainelor compatibile**
   Hainele sunt grupate astfel încât toate articolele din același grup să aibă o temperatură relativ comună de spălare, culoarea și materialul compatibile și aceleași restricții de uscare/albire/călcare.

4. **Generarea recomandării**
   Pentru fiecare grup format, se selectează un program standardizat, în funcție de cel mai restrictiv profil din grup.

Algorithm 1 Gruparea hainelor compatibile

```
Input: lista_haine = [haina_1, haina_2, ..., haina_N]
Output: grupuri_compatibile = []
for all haina in lista_haine do
    Extrage atribute
    Creează profil
end for
Inițializează grupuri_compatibile = []
for all profil in lista_haine do
    grup_gasit ← False
    for all grup in grupuri_compatibile do
        if profilul este compatibil cu toate hainele din grup then
            Adaugă haina în grup
            grup_gasit ← True
            break
        end if
    end for
    if grup_gasit = False then
        Creează un nou grup cu această haină
        Adaugă grupul în grupuri_compatibile
    end if
end for
return grupuri_compatibile
```

Figura 4.10: Algoritmul de grupare

Această abordare deterministă asigură compatibilitatea hainelor în cadrul aceleiași spălări și poate fi extinsă cu noi reguli.

Capitolul 5

Fluxul de utilizare al aplicației

5.1 Autentificarea - pagina de înregistrare și logare

LaundryMate oferă un flux intuitiv de autentificare în aplicație, fiind compus astfel: **ecranul de logare**, unde utilizatorul introduce adresa de email și parola pentru a se autentifica; ecranul de **resetare a parolei**, care permite trimiterea unui email de recuperare către utilizator; **crearea contului**, în care utilizatorul completează toate câmpurile, iar cel de parolă este asociat unui set de validatoare care verifică în timp real îndeplinirea cerințelor de siguranță și etapa de **verificare a email-ului**, unde aplicația trimite automat un mesaj de verificare, iar contul nu poate fi folosit înainte de validarea adresei. În cazul în care utilizatorul nu primește emailul în termen de 60 de secunde, are posibilitatea de a trimite din nou email-ul prin opțiunea ”resend email”.

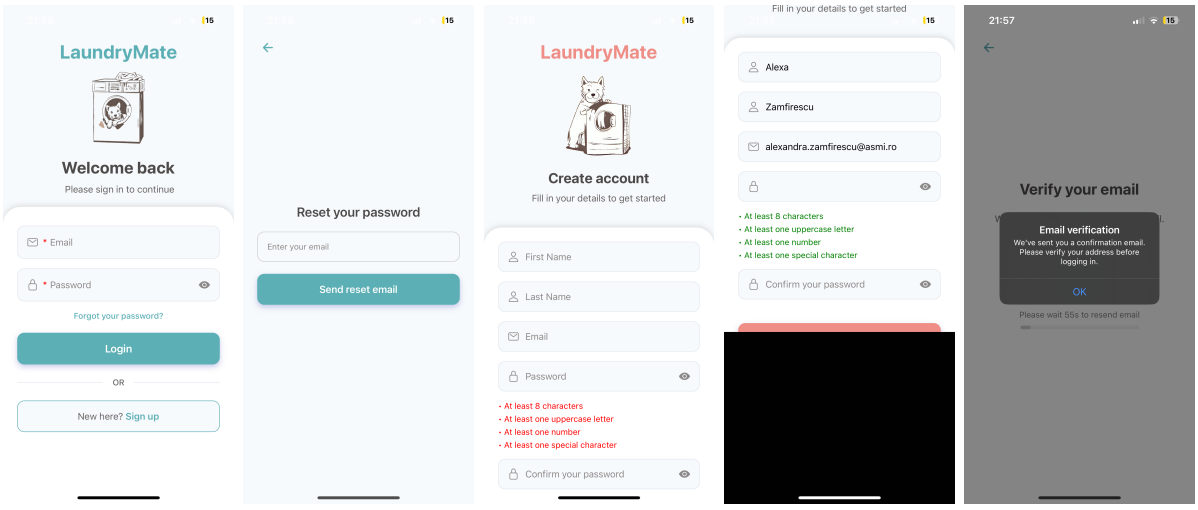


Figura 5.1: Autentificarea în aplicație

5.2 Gestionarea contului și preferințelor utilizatorului

Pe pagina de profil, utilizatorul poate vizualiza și edita informațiile contului, schimba parola, gestiona preferințele de notificare, modifica limba aplicației (ro/en) și comuta între modurile Light/Dark. De asemenea, aplicația oferă opțiunea de deconectare sau de ștergere permanentă a contului.

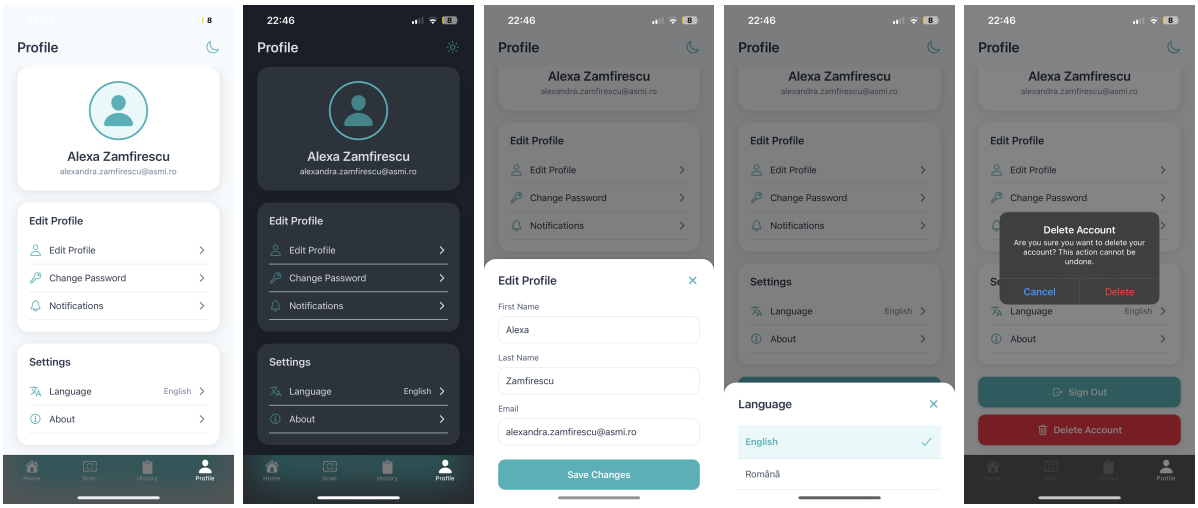


Figura 5.2: Meniul de preferințe al utilizatorului

5.3 Pagina principală

Ecranul principal afișează un mesaj personalizat de bun venit, ”Tip of the Day” generat de LLM pentru oferirea sfaturilor de îngrijire a hainelor, ultima recomandare de spălare cu posibilitatea de reutilizare. De asemenea, sunt afișate statistici personale ale utilizatorului precum numărul de sesiuni generate și de haine scanate, iar în partea dreaptă de sus a paginii este un buton de scanare rapidă, redirecționând utilizatorul direct pe funcția de cameră pentru fotografierea etichetei.



Figura 5.3: Pagina principală

5.4 Scanarea hainelor și generarea recomandării

Utilizatorul poate fotografia o etichetă de întreținere folosind camera sau încărca o imagine din galerie. Imaginea este transmisă către backend, unde este procesată de modelul YOLOv8n, iar simbolurile detectate sunt afișate în interfață. După scanarea mai multor haine, utilizatorul apasă pe butonul de finalizare a scanării, declanșând procesul de grupare a hainelor compatibile și de afișare a recomandării de programe de spălare.

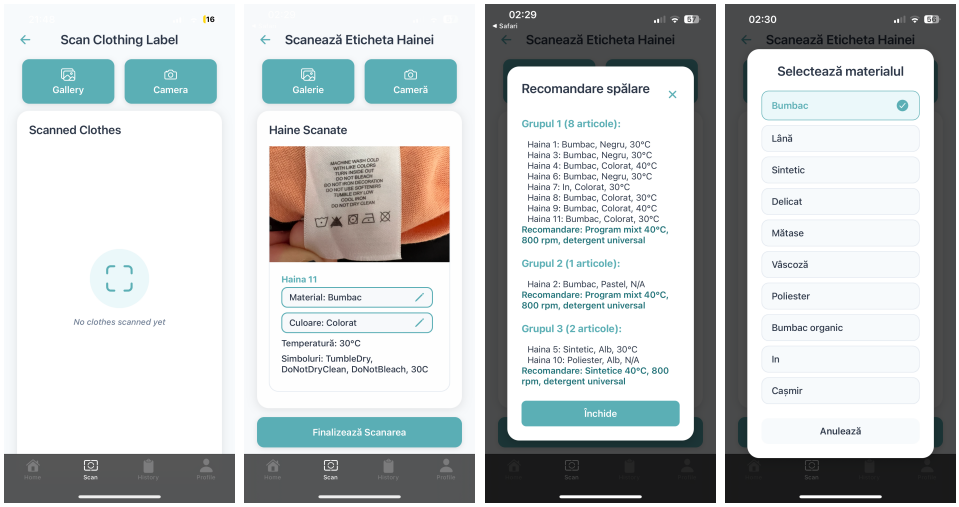


Figura 5.4: Fluxul de scanare și afișare

5.5 Salvarea hainelor în istoric

După generarea recomandării, hainele scanate și grupurile create sunt transmise către backend și stocate în colecția `/garments/` a utilizatorului din Firestore. Aceste date sunt afișate ulterior în pagina de ”Istoric”, unde pot fi consultate, reutilizate sau șterse. Secțiunea include și filtre după dată sau tip de material.

5.6 Notificări și reamintiri automate

Pentru a sprijini utilizatorii în menținerea unei rutine de spălare, aplicația trimite notificări automate în cazul în care nu au fost scanate haine în ultimele 3 zile. Mesajele sunt afișate în interfața mobilă ca alerte push și pot fi oprite din secțiunea de preferințe ale utilizatorului.

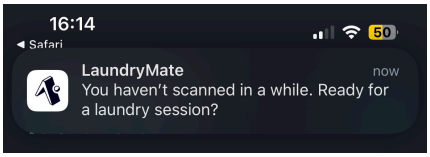


Figura 5.5: Exemplu de notificare primită de utilizator

Capitolul 6

Concluzii

Lucrarea de față a prezentat procesul de dezvoltare a aplicației LaundryMate, un asistent mobil dedicat interpretării simbolurilor de întreținere a textilelor și generării de recomandări utile pentru utilizatori. Scopul principal a fost facilitarea procesului de grupare și configurare a programului de spălare, reducând riscurile de deteriorare a hainelor.

Pentru detecția simbolurilor s-a utilizat modelul YOLOv8n, o arhitectură pe un set de date personalizat. Performanța acestuia a fost comparată cu YOLOv5s, iar rezultatele au evidențiat o acuratețe superioară pentru versiunea aleasă. Procesul de grupare a hainelor a fost implementat printr-o logică decizională deterministă, fără a apela la algoritmi de învățare automată, garantând transparență și control asupra recomandărilor oferite.

În plus, aplicația integrează funcționalități moderne precum notificările push, salvarea unui istoric al hainelor scanate și interacțiunea cu un model LLM pentru generarea de sfaturi zilnice.

Direcții viitoare de dezvoltare

Pe viitor, aplicația poate fi extinsă în mai multe direcții care să vizeze creșterea nivelului de automatizare și personalizare. Una dintre cele mai promițătoare îmbunătățiri constă în dezvoltarea unui **sistem de recomandare complet automat**, capabil să analizeze întregul conținut al coșului de rufe și să genereze sugestii personalizate privind programul de spălare. De asemenea, o direcție tehnologică viabilă constă în **conectarea aplicației la mașini de spălat inteligente**, facilitând transmiterea automată a parametrilor de spălare. Nu în ultimul rând, se are în vedere extinderea funcționalităților din perspectiva accesibilității prin **implementarea unui sistem de feedback audio**, inspirat de aplicații pentru nevăzători precum **Laundry Lens**[16]. LaundryMate evidențiază potențialul aplicațiilor mobile de a automatiza și eficientiza sarcini casnice frecvente, prin îmbinarea tehnologiilor pentru a oferi o experiență de utilizare mai sigură, mai informată și mai confortabilă.

Bibliografie

[1] Jan Kralicek, Lukas Neumann și Jiri Matas, „Care Label Recognition", în *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2019.

[2] Hansi Seitaj și Vinayak Elangovan, „Information Extraction from Product Labels: A Machine Vision Approach", în *International Journal of Artificial Intelligence and Applications (IJAIA)* 15.2 (Mar. 2024).

[3] K. Hemalatha și B. Venkateswarlu, „Developing an Intelligent Washing Machine with a Fuzzy Logic Controller", în *International Research Journal of Multidisciplinary Scope (IRJMS)* 5.3 (2024), pp. 728–737, DOI: 10.47857/irjms.2024.v05i03.0864.

[4] Johannes Bauer, *Laundry Symbols AI Scanner 1.4*, Accesat: 05.06.2025, 2024, URL: https://laundry-symbols-ai-scanner.updatestar.com/en.

[5] Juli Clover, *iOS 17 Photos App Can Tell You What Those Confusing Laundry Symbols Mean*, Accesat: 09.06.2025, 2023, URL: https://www.macrumors.com/2023/06/28/ios-17-laundry-symbol-visual-lookup/.

[6] Meta Platforms Inc., *React Native Documentation*, https://reactnative.dev/docs/getting-started, Accesat: 04.06.2025, 2025.

[7] Expo, *Expo Documentation*, https://docs.expo.dev, Accesat: 10.06.2025, 2025.

[8] Django Software Foundation, *Django Documentation*, https://docs.djangoproject.com/en/4.1/, Accesat: 07.06.2025, 2025.

[9] Celery Project, *Celery 5.3 Documentation*, https://docs.celeryq.dev/en/stable/, Accesat: 31.05.2025, 2025.

[10] Google LLC, *Firebase Documentation – Authentication, Firestore*, https://firebase.google.com/docs, Accesat: 09.06.2025, 2025.

[11] Alexey Bochkovskiy, Chien-Yao Wang și Hong-Yuan Mark Liao, „YOLOv4: Optimal Speed and Accuracy of Object Detection", în *arXiv* abs/2004.10934 (Apr. 2020), Accesat: 09.06.2025, URL: https://arxiv.org/abs/2004.10934.

[12] Ultralytics, *YOLOv8 Documentation*, https://docs.ultralytics.com, Accesat: 31.05.2025, 2024.

[13] Roboflow Inc., *Roboflow Platform Documentation*, https://docs.roboflow.com, Accesat: 06.06.2025, 2024.

[14] Google LLC, *Gemini API – Google Generative Language*, https://ai.google.dev, Accesat: 31.05.2025, 2025.

[15] Glenn Jocher și Ultralytics, *YOLOv5 by Ultralytics*, https://github.com/ultralytics/yolov5, Accesat: 31.05.2025, 2023.

[16] Henshaws, *Laundry Lens: An app to help with clothes washing*, Accesat: 04.06.2025, 2020, URL: https://www.henshaws.org.uk/hints-and-tips/laundry-lens-app-helps-with-laundry/.